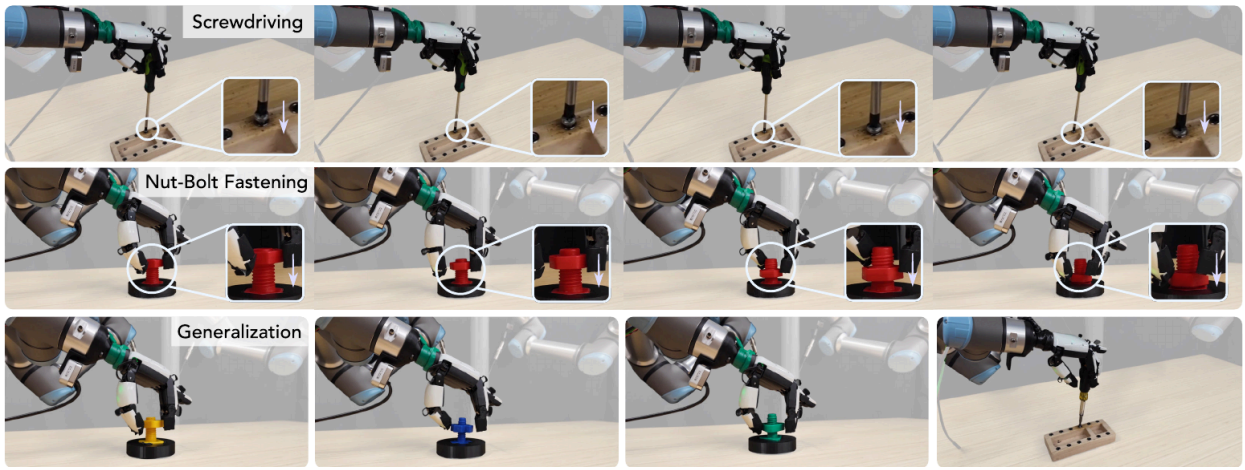
DexScrew: 분석 리포트

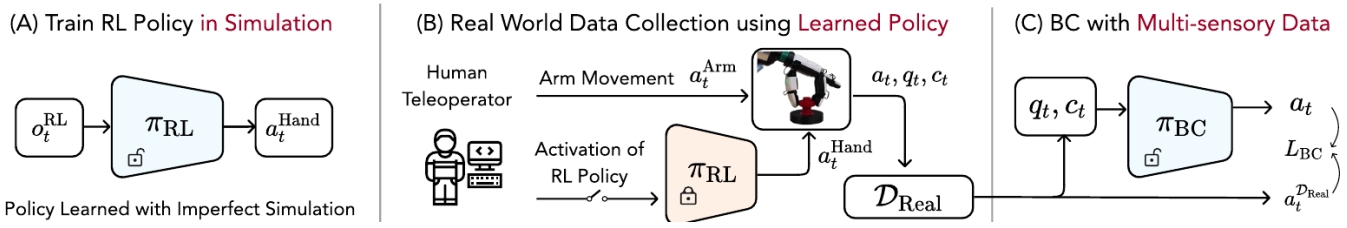
arXiv:2512.02011v2 · UC Berkeley (Hsieh, Hsieh, Wang, Lin, Malik, Sreenath, Qi) · 코드 x-robotics-lab/dexscrew @ 3bde4e3, skill-teleop @ dec60d6, minbc @ b64c53f · 분석일 2026-10-28



논문 Fig. 1: 드라이버 돌리기(위), 너트 조이기(가운데), 처음 보는 물체로의 일반화(아래).

핵심 요약

1. **무엇을 하는가:** 나사산은 시뮬레이션하지 않고, 너트를 회전 관절 하나로 근사해 RL로 손가락 회전 gait만 학습한다. 그 정책을 텔레옵의 "회전 버튼"으로 써서 실제 촉각 데이터를 모으고, BC로 팔과 손가락을 함께 제어한다. 결과는 너트 진행률 95–98%, 드라이버 95%다.
2. **핵심 설계:** 시뮬레이션이 빼먹은 것은 사실상 **회전-하강 결합(z = pθ/2π) 하나**이고, 사람과 BC가 그것을 메운다. "불완전한 시뮬레이션"은 "동작을 결정하는 운동학만 맞으면 된다"는 뜻에 가깝다.
3. **통계 재계산:** Table I의 16칸은 모두 8회전(12.5% 단위)으로 정확히 재구성되므로 숫자는 일관된다. 그러나 **촉각+히스토리 vs 히스토리는 너트별로 유의하지 않다**(p 0.16–0.37). 촉각의 효과는 평균보다 **실패 꼬리 제거**(75% 미만 시행 0/40 vs 5–8/40)로 드러난다.
4. **공개 코드 버그:** privileged info 인덱스가 0부터 다시 시작해서(상위 코드 penspin은 9), oracle은 **질량·마찰·스케일을 보지 못한다.** 그러므로 Fig. 7의 "privileged info가 중요하다"는 결과는 동역학 식별이 아니라 회전 상태 관측의 효과일 수 있다.



논문 Fig. 2: (A) 단순화 시뮬레이션 RL → (B) RL 기술을 쓰는 텔레옵 → (C) 촉각 BC.

01. 논문 분석: DexScrew (arXiv:2512.02011)

- 제목: *Learning Dexterous Manipulation Skills from Imperfect Simulations*
- 저자: Elvis Hsieh*, Wen-Han Hsieh*, Yen-Jen Wang*, Toru Lin, Jitendra Malik, Koushil Sreenath †, Haozhi Qi † (UC Berkeley)
- 공개: arXiv v1 2025-12-01, v2 2026-02-25 (분석은 v2 기준), 라이선스 CC BY 4.0
- 코드: RL `x-robotics-lab/dexscrew` @ `3bde4e3`, 텔레옵·배포 `x-robotics-lab/skill-teleop` @ `dec60d6`, BC `x-robotics-lab/minbc` @ `b64c53f` (README가 지정한 논문 기준 버전). 세 저장소 모두 MIT 라이선스입니다.
- 프로젝트: https://dexscrew.github.io

---

1. Abstract (한 줄 요약과 주장)

한 줄 요약: 나사산처럼 시뮬레이션하기 어려운 접촉은 시뮬레이터에서 아예 빼고, **회전 동작만** 단순화된 모델(회전 관절 하나)로 RL 학습합니다. 그 정책을 텔레옵의 "손가락 회전 버튼"으로 써서 실제 촉각 데이터를 모으고, BC로 팔과 손가락을 함께 제어하는 촉각 정책을 학습합니다.

논문의 주장은 세 가지입니다.

1. 접촉이 많은 조작의 **기본 동작(finger gait)은 정확한 물리 모델 없이도 학습된다.** 너트와 볼트를 회전 관절로 근사해도 실제로 옮겨 쓸 수 있는 회전 gait가 나옵니다.
2. **시뮬레이션 정책을 기술 단위(skill primitive)로 쓰는 텔레옵이 다지 손 데이터 수집을 쉽게 만든다.** 사람은 손목만 조종하고 손가락은 RL 정책에 맡깁니다.
3. 촉각과 관측 히스토리를 쓰는 BC 정책은 직접 sim-to-real보다 진행률이 크게 높고, 처음 보는 너트 모양과 외란에도 강건하다.

---

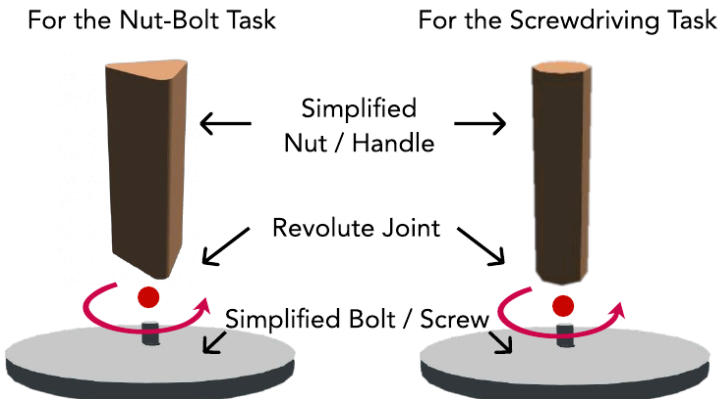
2. Method

2.1 3단계 구조 (Fig. 2)

```
(A) 시뮬레이션 RL  — IsaacGym, 8,192 env, XHand 12-DoF, 200 Hz sim / 20 Hz 제어
    단순화 물체 = 고정 원기둥 받침 + 회전 관절(revolute)로 연결된 너트·손잡이 (나사산 없음, 축 방향 이동 없음)
    oracle(PPO, privileged info) → sensorimotor(proprio 30스텝 히스토리로 latent 추정, DAgger식 증류)
                │  JIT로 변환한 회전 정책 π_RL
                ▼
(B) 기술 기반 텔레옵 — Quest 2 컨트롤러: 조이스틱으로 손목 평행이동·yaw·pitch, 버튼으로 손목 상하 이동, 트리거로 π_RL 켜기
    기록: a_t = [a_t^Hand(12, π_RL 출력 또는 현재 관절), a_t^Arm(6)], q_t(18), 촉각 c_t ∈ R^{5×120×3}
                ▼
(C) BC — 입력: 최근 K=5 스텝의 (q, c), 촉각은 MLP로 압축 → hourglass encoder → H=16 action chunk
```

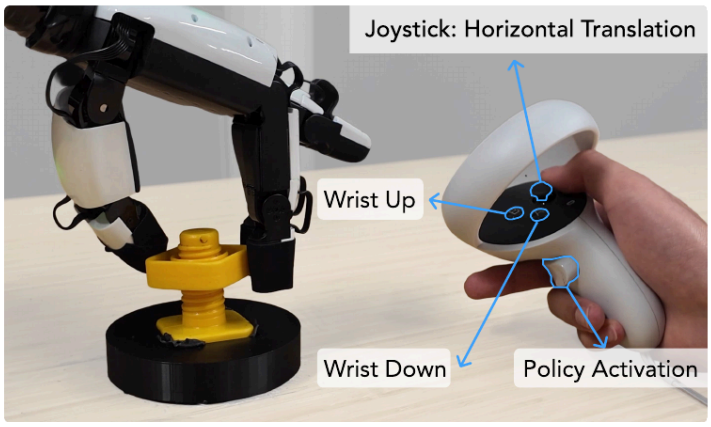
2.2 시뮬레이션 RL의 설계

- **단순화 물체 (Fig. 3):** 너트 작업에는 두꺼운 삼각 너트를 씁니다. 두꺼운 이유는 아래에서 밀어 올리는 편법을 막기 위해서입니다. 드라이버 작업에는 8각과 12각 손잡이를 씁니다. 저자들은 학습 물체의 모양이 gait의 성격(높은 clearance, 보수적인 동작)을 결정한다고 설명합니다.
- **행동:** 상대 목표 a_t^Hand = η·f(o_t) + a_{t−1}^Hand이고 PD 제어기로 토크를 만듭니다. 코드상 η는 0.05(드라이버), 0.04167(너트)이고 `clipActions 1.0`입니다. 따라서 관절 목표 속도는 20 Hz에서 최대 약 1.0 rad/s와 0.83 rad/s로 묶입니다.
- **보상:** 다음 항들의 가중합입니다(가중치는 Table IV).
  - 회전 속도 보상(±4 rad/s로 clip)
  - 근접 보상(엄지·검지 거리)
  - 토크 페널티와 일(work) 페널티
  - 초기 자세 이탈 페널티
  - 과속 페널티(10 rad/s 초과. 드라이버는 7.5에서 15로 커리큘럼)
- **종료 조건:** 손가락이 물체에서 멀어짐(너트 7 cm, 손잡이 10 cm), 회전 정체(3–3.5 s), 무접촉 지속
- **랜덤화 (Table V):** 마찰 계수 [0.5, 8.0], 질량 [0.04, 0.06] kg, 스케일, PD 게인, 관측·행동 잡음, 확률 0.25의 외력



논문 Fig. 3: 너트와 손잡이를 회전 관절 하나로 근사한 단순화 물체.

2.3 기술 기반 텔레옵



논문 Fig. 4: Quest 2로 손목을 조종하고 회전 기술을 켜는 텔레옵 인터페이스.

- 사람이 직접 하는 일은 **손목 위치를 맞추고 회전 기술을 켜는 것뿐**입니다. 사람과 로봇 손의 형태 차이 때문에 생기는 손가락 텔레옵의 어려움을 이렇게 우회합니다.
- 나사산 때문에 생기는 **축 방향 하강은 시뮬레이션에서 일부러 뺐고, 사람의 팔 조작이 이것을 보완합니다.** 이 부분이 핵심 설계입니다.

- 수집량: 너트 작업은 사각·삼각 너트 각 50회(약 80 s/회), 드라이버 작업은 72회(120–180 s/회)입니다.

2.4 BC

- 비전은 쓰지 않고 관절 위치와 촉각만 씁니다. 손실은 action chunk 전체의 L2이고, Adam으로 200 epoch 학습합니다.
- 배포 코드는 겹치는 chunk를 **균등 평균하는 temporal ensembling**(`temporal_ensemble_mode="avg"`)을 씁니다. 논문 본문에는 언급이 없습니다.

---

3. Experiments

3.1 너트-볼트 (Table I, 조건당 10회)

촉각	히스토리	사각	삼각	육각(미학습)	십자(미학습)
		63.8±33.1	30.0±39.6	75.0±29.5	63.8±33.1
✓		62.5±33.9	66.3±27.7	75.0±16.7	82.5±10.5
	✓	87.5±20.4	80.0±32.9	85.0±32.2	100±0
✓	✓	**97.5±7.9**	**96.3±8.4**	**95.0±10.5**	98.8±4.0

직접 sim-to-real의 한계: 너트는 돌지만 내려가지 않습니다. 팔이 움직이지 않고 나사산도 시뮬레이션되지 않았기 때문이고, 그래서 수치를 보고하지 않았습니다.

3.2 드라이버 (Table II, 조건당 10회)

방법	진행률 (%)	시간 (s)
직접 sim-to-real	41.6±26.2	완주 없음
Expert replay (수집 데이터 open-loop 재생)	50.8±19.3	완주 없음
BC (없음)	69.2±35.3	266±92
BC + 촉각	67.6±35.7	264±77
BC + 히스토리	87.5±18.6	195±45
BC + 촉각 + 히스토리	**95.0±13.2**	**188±25**

3.3 시뮬레이션 ablation (Fig. 7, 시드 5개)

privileged info를 쓴 oracle이 가장 좋았고, asymmetric actor-critic(critic만 privileged)이 그다음, privileged info를 전혀 쓰지 않은 경우가 가장 나빴습니다. 학습을 더 오래 해도 격차가 줄지 않았다고 보고합니다.

3.4 해석: 수치를 어떻게 읽을 것인가 (`03_analysis.md` §1에서 재계산)

- **촉각+히스토리가 히스토리만 쓴 경우보다 낫다는 주장은 너트별로는 유의하지 않습니다.** Welch p는 0.16–0.37이고 드라이버는 0.31입니다. 4종을 합친 40회 기준으로는 평균 차이가 경계선(순열검정 p≈0.04)이고, 순위 기준(Mann-Whitney)으로는 유의하지 않습니다(p 0.10–0.36).
- **촉각의 효과는 "실패 꼬리 제거"로 보는 편이 정확합니다.** 진행률 75% 미만인 시행이 히스토리만 쓰면 40회 중 5–8회이고, 촉각+히스토리는 0회입니다(Fisher p 0.005–0.055). 평균을 올린다기보다 최악의 경우를 없앱니다.

- **히스토리의 효과가 더 뚜렷합니다.** 삼각 너트에서 p=0.007, 십자 너트에서 p=0.007입니다.
- **촉각 단독의 효과는 거의 없습니다.** 너트(사각·육각)와 드라이버 모두 차이가 0에 가깝습니다(드라이버 p=0.92). 저자들도 "히스토리가 없으면 촉각의 이득이 제한적"이라고 썼습니다.
- **"BC가 데이터를 만든 정책보다 낫다"(69.2 vs 50.8)는 주장도 n=10에서는 유의하지 않습니다**(p=0.17). 저자들이 근거로 든 filtered BC(성공 시행만 학습)도, 공개된 데이터 변환 코드에는 성공 여부로 거르는 단계가 없습니다(`04` D5).

---

4. Limitations

4.1 저자가 밝힌 한계

- 텔레옵에 여전히 사람이 필요합니다.
- 너트가 이미 볼트에 걸려 있고 드라이버가 이미 나사에 꽂혀 있는 상태에서 시작합니다(장기 조립은 다루지 않음).
- 비전과 정밀한 힘·토크 센싱을 쓰지 않습니다.
- 과제가 두 개뿐입니다.

4.2 분석하면서 추가로 발견한 한계와 의문점

1. **공개 코드의 privileged info가 논문 설명과 다릅니다.** 인덱스가 겹치는 버그 때문에 oracle은 물체 질량·마찰·스케일·각속도·접촉 여부를 보지 못합니다(`04` D1, 상위 코드 penspin과 대조해 확인). 따라서 Fig. 7의 "privileged info가 중요하다"는 결과는 "**동역학 파라미터 식별**"(RMA식 해석)이 아니라 "**너트 회전 상태와 손끝 상태 관측**"의 효과로 다시 읽어야 할 수 있습니다. 다만 논문 실험에 이 코드 버전이 쓰였는지는 확인할 수 없습니다.
2. **통계적 검정력이 낮습니다.** 조건당 10회로는 $\sigma \approx 20$, 효과 10%p를 80% 검정력으로 검출할 수 없습니다(필요 $n \approx 63$). 신뢰구간이나 검정이 하나도 보고되지 않았습니다.
3. **진행률 지표의 정의가 불완전합니다.** "필요한 회전 수"가 명시되지 않았습니다. Table I의 16칸은 모두 8회전(12.5% 단위) 가정과 정확히 맞으므로 데이터는 내부적으로 일관됩니다. 드라이버 표는 단위가 일정하지 않아 연속값으로 측정한 것으로 보입니다.
4. **수치 불일치가 있습니다.** 본문은 oracle 학습을 1.5×10^9 step이라고 하고, Table VI와 Fig. 7의 x축은 3×10^9입니다. Table III은 손끝 2개(엄지·검지) 기준 97차원이라고 하지만, 코드는 손끝 5개 기준입니다(드라이버 84차원, 너트 127차원).
5. **보상에 정의되지 않은 항이 있습니다.** Table IV의 λ_pc-z(−1.0)는 본문에 정의가 없습니다. 코드를 보면 물체 point cloud의 z 범위에 대한 페널티인데, 물체 받침이 고정되어 있어서 **행동과 무관한 상수**가 됩니다(`04` D3).
6. **"시뮬레이션이 불완전해도 된다"는 범위가 좁습니다.** 실제로 빠진 것은 나사산이 만드는 회전-하강 결합 하나이고, 이 결합을 사람이 대신 해 줍니다. 더 복잡한 접촉(마찰 변화, 걸림)을 다루는 과제로 일반화될지는 보여주지 않았습니다.

03. 재계산과 구조 분석

GPU 없이 할 수 있는 검증만 했습니다. 스크립트와 결과 파일은 다음과 같습니다.

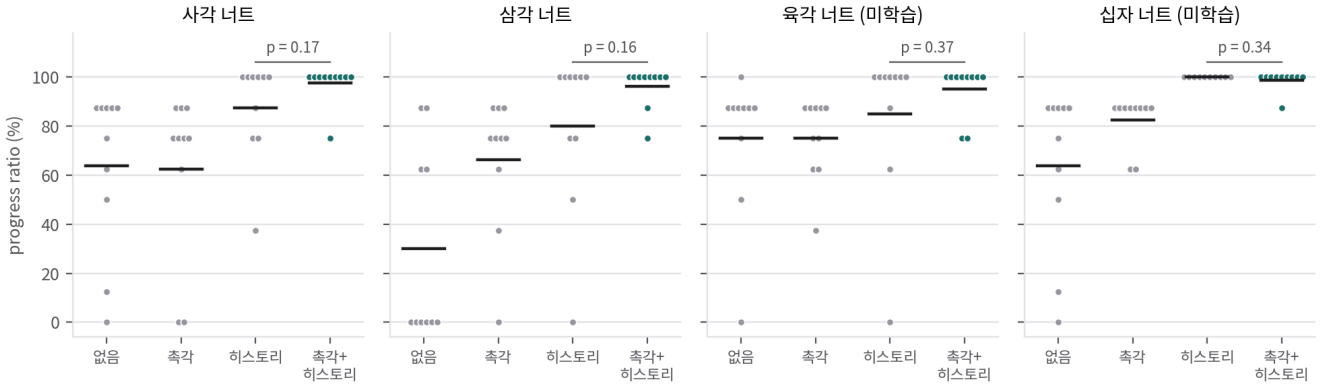
- `analysis/stats_check.py` → `stats_check_results.json`
- `analysis/plot_stats.py` → `report/figures/stats_nut_dots.png`
- `analysis/priv_layout_check.py` → `priv_layout_{screwdriver,nutbolt}.json`

---

1. 실험 표의 통계 재계산

1.1 표 값은 내부적으로 일관된다 (GRIMMER식 재구성)

- **방법:** 시행 10회의 진행률이 g% 단위 값만 가질 수 있다고 가정합니다. 그런 값 10개를 고르는 모든 조합을 전수 탐색해서, 표의 (평균, 표본 표준편차)를 소수 둘째 자리까지 재현하는 조합이 있는지 확인했습니다.
- **결과:** Table I의 16칸이 **모두 g = 12.5%(8회전 중 k회전)로 재구성됩니다.** 그중 촉각+히스토리 4칸과 히스토리/십자 1칸은 해가 **유일**합니다. 예를 들어 사각 너트의 촉각+히스토리(97.5±7.91)는 "100% 9회, 75% 1회"로만 만들어집니다.
- **해석:** 숫자가 만들어진 흔적은 없습니다. 다만 논문은 "필요한 회전 수"를 적지 않았고, 여기서 8회전은 추론한 값입니다.
- **드라이버 표(Table II):** 한 가지 단위로 맞지 않으므로(예: 41.6과 87.5) 연속값으로 측정한 것으로 보고 재구성하지 않았습니다.



점: 표의 평균±SD를 만족하도록 재구성한 시행 10회 (12.5% = 8회전 중 1회전 가정, 해가 여러 개면 하나만 표시) 검은 선: 평균 p: Welch t-test (히스토리 vs 촉각+히스토리)

1.2 유의성

비교	너트별 Welch p	4종 합산 (n=40)
촉각+히스토리 vs 히스토리	0.17 / 0.16 / 0.37 / 0.34	순열 p 0.042–0.046, Mann-Whitney p 0.10–0.36
촉각 vs 없음	0.93 / **0.03** / 1.00 / 0.12	
히스토리 vs 없음	0.07 / **0.007** / 0.48 / **0.007**	
드라이버: 촉각+히스토리 vs 히스토리	0.31	
드라이버: BC(없음) vs expert replay	0.17	

- p값에 범위가 있는 이유는 재구성이 유일하지 않은 칸이 있어서, 가능한 조합 18개를 모두 계산했기 때문입니다.

- **실패 꼬리:** 진행률 75% 미만인 시행 수는 촉각+히스토리가 **0/40**, 히스토리가 **5–8/40**입니다(Fisher p 0.005–0.055).

결론:

- 촉각은 평균적인 성능보다 **최악의 경우(접촉을 잃고 회전이 멈추는 시행)** 를 없애는 데 기여합니다. 논문 Fig. 5의 정성적 설명(촉각이 있으면 엄지와 검지가 번갈아 닿는 패턴이 유지됨)과도 맞습니다.
- 히스토리는 더 큰 폭으로, 더 일관되게 기여합니다.
- "촉각이 필수적"이라는 주장은 현재 표본 크기로는 통계적으로 뒷받침되지 않습니다. 평균 차이 10%p를 80% 검정력으로 검출하려면 조건당 약 63회(σ=20 기준)가 필요하고, σ=13·차이 7.5%p라면 약 48회가 필요합니다.

---

2. Privileged info 인덱스 겹침의 영향

코드의 규칙을 그대로 재현해 보면, oracle 정책이 실제로 받는 벡터에서 각 항목이 살아남는 비율은 다음과 같습니다 (`04` D1).

항목	드라이버	너트
물체 위치, 스케일, 질량, 마찰	**0 / 6**	**0 / 6**
물체 무게중심	2 / 3	2 / 3
물체 각속도, hand scale, 너트 접촉	**0 / 5** (값을 쓰는 코드가 아예 없음)	**0 / 5**
물체 자세·선속도, 손끝 상태(5개), 너트 위치·회전각·회전속도, 반발계수, 나사 마찰	전부 보임	전부 보임
PD 게인, 손 관절·자세·위치	사용 안 함	전부 보임

함의:

- 논문은 privileged info의 대표 예로 "질량, 마찰, 무게중심"을 들고 RMA·Hora 계열의 설명을 따르지만, 공개 코드의 oracle은 **도메인 랜덤화한 동역학 파라미터 대부분을 보지 못합니다.**
- 그럼에도 oracle이 잘 학습된다는 것(Fig. 7)은, 성능 차이의 원천이 **너트 회전각·속도 같은 과제 진행 상태와 손끝 상태**라는 뜻일 가능성이 큽니다.
- 그렇다면 sensorimotor 정책이 30스텝 히스토리로 추정해야 하는 것도 "물리 파라미터"가 아니라 "현재 회전 진행 상황"입니다. 이 해석은 BC에서 히스토리 효과가 크게 나온 것과도 맞습니다.

---

3. 단순화 시뮬레이션이 실제로 빼먹은 것: 회전-하강 결합

- **나사의 운동학:** 실제 나사는 z = (p / 2π)·θ (p는 피치) 관계로 회전과 축 방향 이동이 묶여 있습니다. 회전 관절 모델은 이 결합을 끊어 버리므로, 시뮬레이션 정책은 제자리 회전만 배웁니다.
- **역할 분담:** 논문의 설계는 이 결합 하나를 **사람(텔레옵)과 BC가 대신 맡게** 한 것입니다. 사람이 손목을 내리면, BC는 촉각 패턴을 보고 언제 내릴지를 학습합니다.
- **대안:** 이 결합은 시뮬레이터에서 아주 싸게 넣을 수 있습니다. 예를 들어 MuJoCo는 `<equality><joint joint1="nut_z" joint2="nut_rot" polycoef="0 k 0 0 0"/>` 한 줄로 직선형 결합을 표현합니다. 그러면 "회전 + 하강"까지 시뮬레이션에서 학습한 정책을 텔레옵 기술로 쓸 수 있어, 사람의 부담이 더 줄어들 것으로 예상합니다(검증되지 않은 가설).

- **"불완전한 시뮬레이션"의 정확한 의미:** 정리하면 이 논문은 "모든 것이 부정확해도 된다"가 아니라, **"동작을 결정하는 운동학만 맞으면, 접촉 세부와 센서는 실제 데이터로 채울 수 있다"** 는 주장에 가깝습니다.

---

4. 행동 파라미터화와 텔레옵 데이터의 의미

- **RL 행동:** a_t = η·f(o) + a_{t−1}은 목표 관절각을 적분하는 형태입니다. |f| ≤ 1이므로 목표 속도는 η × 20 Hz로 묶입니다(드라이버 1.0 rad/s, 너트 0.83 rad/s). 이 상한이 실제 하드웨어에서 안전 장치 역할도 합니다.
- **BC 레이블:** BC가 기록하는 a^Hand는 **절대 목표 관절각**(π_RL의 출력, 정책이 꺼져 있으면 현재 관절각)입니다. 따라서 BC는 RL 정책의 증분 구조를 그대로 상속하지 않고 절대 위치 chunk를 예측합니다. 배포할 때 겹치는 chunk를 균등 평균하므로, 출력이 RL 정책보다 매끄러워지는 효과가 있을 것입니다.
- **"BC > expert replay"의 해석:** 논문은 filtered BC를 근거로 들지만, 공개된 변환 스크립트에는 성공 여부로 거르는 단계가 없습니다(앞뒤 구간 자르기만 있음). 가장 단순한 설명은 **open-loop 재생과 closed-loop 정책의 차이**입니다. replay는 같은 행동열을 다른 초기 상태에서 재생하므로 오차가 누적되고, BC는 관측에 반응합니다. 다만 두 값의 차이 자체도 p=0.17로 유의하지 않습니다.

04. 코드 검증: 논문 ↔ 구현 대조

- **대상:** `dexscrew` @ `3bde4e3`, `skill-teleop` @ `dec60d6`, `minbc` @ `b64c53f` (README가 지정한 논문 기준 버전)
- **범위:** 코드 읽기와 정적 재현만 했습니다. IsaacGym 학습과 실제 로봇 실행은 하지 않았습니다.

1. 모듈 매핑

논문	구현 위치	일치 여부
단순화 물체 (회전 관절, 받침 고정)	`assets/screw/*`, `xhand_hora.py:1180` `fix_base_link=True`	✅
행동 a = η·f + a_{t−1}, PD 토크 제어	`configs/task/*.yaml` `action_scale 0.05 / 0.04167`, `pgain 3`, `dgain 0.01`, `controlFrequencyInv 10` (20 Hz)	✅
보상 항과 가중치 (Table IV)	`xhand_hora.py:672-730`, `compute_hand_reward`	✅ 가중치 일치. 단 D3 참조
과속 페널티 커리큘럼 7.5→15 rad/s	드라이버 yaml `angvelPenaltyThres: [7.5, 15.0, 3e7, 6e7]`	✅
종료 조건 (거리, 정체, 무접촉)	`check_termination` (`xhand_hora.py:878-915`)	✅ 거리 임계값은 스크립트에서 덮어씀: oracle 너트 0.07, 드라이버 0.1, student 드라이버 0.15. 그 밖에 나사 관절 한계 근처에서 리셋하는 조건이 추가로 있음
랜덤화 (Table V)	yaml `randomization`	✅ 수치 일치
PPO 하이퍼파라미터 (Table VI)	`configs/train/*.yaml`	✅ (lr 5e-3, horizon 12, minibatch 16384, γ 0.99, λ 0.95, clip 0.2, kl 0.02). 단 `max_agent_steps 1e10`이라 학습 종료 시점은 수동
Sensorimotor 증류 $L = \|a-\hat{a}\|^2 + \|z-\hat{z}\|^2$	`algo/ppo/padapt.py:159-170`	✅ (행동은 [−1,1]로 clip한 뒤 비교)
Student env 48개 (Table VII)	`scripts/*_student_padapt.sh` `numEnvs=48`	✅
촉각 5×120×3 = 1800	`minbc/configs/base.py` `xhand_tactile_input_dim 1800`	✅
BC K=5, H=16, hourglass, 200 epoch	`minbc` 기본값은 `obs_horizon 1`, `act_horizon 8`, `action_decoder mlp`, `num_epoch 30`	⚠️ 논문 설정을 담은 실행 스크립트나 설정 파일이 공개되지 않음

2. 불일치와 버그

#	내용	영향
D1	**Privileged info 인덱스 겹침 (버그).** `xhand_hora.py:1024-1057` 에서 앞의 5개 항목(물체 위치·스케일·질량·마찰·무게중심)을 0–9 칸에 고정한 뒤, 나머지 항목을 `start_index = 0` 부터 다시 배치합니다. 상위 코드 penspin(`allegro_hand_hora.py:836`)은 `start_index = 9` 입니다. 매 스텝 동적 항목(물체 자세·선속도 등)이 0–9 칸을 덮어쓰므로, oracle은 **질량·마찰·스케일·위치를 보지 못합니다.**	`03` §2. 논문 Table III과 Fig. 7의 해석이 달라질 수 있습니다. 논문 실험에 이 버전이 쓰였는지는 확인할 수 없습니다.
D2	`obj_angvel` , `hand_scale` , `nut_contact` 는 차원만 잡혀 있고 `_update_priv_buf` 호출이 없습니다. 다른 항목에 덮어써지지 않은 칸은 항상 0입니다.	Table III에 있는 "너트 접촉 지시자"와 "물체 각속도"는 실제로는 전달되지 않습니다.
D3	**λ_pc-z 항은 상수입니다.** 물체 point cloud z 범위가 3 cm를 넘으면 페널티를 주는데 (`xhand_hora.py:714-717`), 물체 받침이 고정되어 있어서 point cloud가 에피소드 안에서 회전하지 않습니다(드라이버는 리셋 때 ±5° 기울기만 무작위). 즉 행동과 무관한 상수 보상입니다. 같은 위치에 계산만 하고 쓰지 않는 `reverse_penalty` 도 있습니다.	학습에 미치는 영향은 작습니다. 상수 음의 보상은 종료를 약하게 유도할 수 있지만 회전 보상(가중치를 곱하면 최대 10–24/step)에 비하면 작습니다. 본문에는 정의가 없습니다.
D4	**Oracle이 물체 point cloud(100점)를 입력으로 받습니다** (`use_point_cloud_info: True` , `point_mlp [32,32,32]`). 본문과 Table III에는 없고, 저장소 README에만 "privileged information and point-clouds"라고 적혀 있습니다.	물체 모양 정보가 oracle에 직접 들어갑니다. 서로 다른 너트 모양에 따른 gait 차이의 원천일 수 있습니다.
D5	BC 데이터 변환(`skill-teleop/scripts/convert_to_bc_dataset.py`)에는 **성공 시행만 남기는 필터가 없습니다.** 폴더별 앞뒤 구간 자르기와 깨진 파일 제외만 합니다.	논문이 "BC > replay"의 근거로 든 filtered BC 설명이 공개 파이프라인과 맞지 않습니다(수동 선별 가능성은 배제할 수 없음).
D6	배포는 겹치는 chunk를 **균등 평균**하는 temporal ensembling을 씁니다(`run_env.py:356` , `zmq_server_client.py:139`).	본문에 없는 장치입니다. 매끄러움에 기여할 수 있습니다.
D7	Oracle 학습 step 수: 본문은 1.5×10^9, Table VI와 Fig. 7은 3×10^9입니다.	표기 불일치
D8	Table III은 손끝 2개 기준 97차원이지만, 코드는 `fingers_num = 5` 라서 드라이버 84차원, 너트 127차원입니다(`analysis/priv_layout_*.json`).	표기 불일치
D9	`screw_joint_friction` 은 항상 0.2로 고정해서 기록합니다(`xhand_hora.py:285`).	정보량이 없는 privileged 항목입니다.

3. 재현할 때의 요구 사항

- **RL:** IsaacGym Preview(구형 CUDA, Python 3.8 계열)와 GPU 1장. 논문 기준으로 하루 이내에 학습됩니다. 자세한 내용은 `dexscrew/docs/install.md` 에 있습니다.
- **텔레옵과 BC 배포:** UR5e, XHand(촉각 내장), Quest 2, RealSense가 필요합니다. 하드웨어가 없으면 재현할 수 없습니다.
- **BC 학습:** GPU가 있어야 하고 수집 데이터가 필요한데, 데이터셋은 **공개되지 않았습니다.**

02. 선행연구와 계보

번호 [n]은 논문의 참고문헌 번호입니다. 각 논문을 별도로 열어 검증하지는 않았습니다.

1. 계보: Malik·Qi 그룹의 손 안 회전 연구

```
Hora [5] (CoRL 2022)            Allegro, 회전 관절 없는 손 안 물체 회전, oracle → proprio 히스토리 적응(RMA [11] 방식)
  └ General in-hand rotation with vision and touch [17] (CoRL 2023)   시각과 촉각 추가 (촉각은 proxy 표현)
  └ Lessons from learning to spin pens [13] = penspin (CoRL 2024)    시뮬레이션 oracle → 실제 replay로 데이터 수집 → BC로 미세조정
  └ Twisting lids off with two hands [44] (CoRL 2024)                병뚜껑을 회전 관절과 휴리스틱 마찰로 근사
  └ Learning visuotactile skills with two multifingered hands [32]   텔레옵 + 촉각 BC
  └ ★ DexScrew                                                       회전 관절로 단순화한 RL 기술 + 그 기술을 쓰는 텔레옵 + 촉각 BC
```

코드 계보: README에 따르면 DexScrew 코드는 penspin, Hora, IsaacGymEnvs를 기반으로 합니다. privileged info 인덱스 버그(`04` D1)는 penspin의 `start_index = 9`가 `0`으로 바뀐 데서 생겼습니다.

penspin과의 차이: penspin [13]도 "시뮬레이션 정책으로 실제 데이터를 모으고 BC로 학습"하는 구조였지만, 그때는 시뮬레이션 정책을 그대로 **replay**했습니다. DexScrew는 정책을 **사람이 켜고 끄는 기술 버튼**으로 만들고 팔은 사람이 움직이게 해서, 시뮬레이션에 없는 축(하강)을 사람이 채우도록 했습니다.

2. 구성 요소별 출처

역할	선행연구	이 논문에서의 쓰임
Privileged 학습	Learning by Cheating [45], RMA [11]	oracle → sensorimotor 증류
증류	DAgger [47]	행동 + latent 손실
RL	PPO [46], IsaacGym [6]	8,192 env
도메인 랜덤화	Peng et al. [7], OpenAI [1]	외력 2.0m, 확률 0.25 (OpenAI [1]의 설정을 따름)
BC	ACT [21], Diffusion Policy [20]	action chunking (H=16)
정규화	LBM [50] arXiv:2507.05331	관측 정규화 방식
네트워크	Stacked Hourglass [49]	hourglass encoder
해석	Filtered BC [51] (Kumar et al., ICLR 2022)	BC가 replay보다 나은 이유로 인용 (`03` §4)
비교	Asymmetric actor-critic [52] arXiv:1710.06542	Fig. 7 baseline

3. 비슷한 문제를 다른 방법으로 푼 연구 (논문 Related Work 기준)

방법	접근	DexScrew와의 차이
DexNDM [36] arXiv:2510.08556	실제 상호작용으로 관절별 신경 동역학 잔차 모델을 만들어 sim-to-real 간극을 보정	동역학을 보정하는 쪽. DexScrew는 동역학을 고치지 않고 데이터로 우회함
DexterityGen [37] arXiv:2502.04307	시뮬레이션 정책을 안정화 제어기로 쓰고 텔레옵을 결합	촉각을 쓰는 자율 정책을 만들지 않음
Kumar et al. [38]	확산 기반 접촉 탐색 + 궤적 최적화로 드라이버 회전	모델 기반
FORGE [39]	힘 기반 탐색으로 sim-to-real, 평행 그리퍼	다지 손이 아니고 재파지가 없음
Transic [40]	사람의 개입으로 sim-to-real 정책을 실제에서 보정	단순한 엔드이펙터
Sim-and-real co-training [41]	시뮬레이션과 실제 데이터 공동 학습	비전 기반

4. 참고문헌에서 발견한 문제

- **[42]의 서지 정보가 본문과 맞지 않습니다.** 본문은 "RLPD [42]"라고 인용하지만, [42]는 Zhang et al., "Traversability-aware legged navigation by learning from real-world visual data"(T-RO 2025)로 적혀 있습니다. 흔히 RLPD라고 부르는 논문은 Ball et al.의 오프라인 데이터 활용 온라인 RL 연구입니다(*제 사전 지식 기준이라 확인이 필요합니다*). 어느 쪽이든 본문의 약칭과 참고문헌 항목이 맞지 않습니다.

5. 논문이 다루지 않았지만 관련 있는 것 (내 판단)

- **MuJoCo의 joint equality 제약:** 나사의 회전-하강 결합을 시뮬레이터에서 싸게 표현할 수 있는 방법입니다(`03` §3).
- **꼬리 위험 지표(CVaR, 최악 k%)로 평가하기:** 촉각의 효과가 평균보다 꼬리에서 드러나므로, 이런 지표로 보고하는 편이 더 정확했을 것입니다.
- **같은 저장소 군의 Choice Policy:** minbc가 지원하는 다중 제안(multi-proposal) BC입니다. 이 논문에서는 쓰지 않았습니다.